

CSD3009 - DATA STRUCTURES AND ANALYSIS OF ALGORITHMS

DR.S.DEVARAJU

Module 1

Introduction: Overview of Data Structures – A Philosophy of Data Structures – The Need for Data Structures – Cost and Benefits – Abstract Data Types - complexity analysis – Best, Worst, and Average Cases – Asymptotic Analysis - Arrays, Linked Lists and Recursion: Using Arrays – Lists – Array based List Implementation – Linked Lists – LL ADT – Singly Linked List – Doubly Linked List – Circular Linked List – recursion- linear, binary, and multiple recursions.

<https://www.javatpoint.com/data-structure-tutorial>

Introduction

Definition

- Data structure is representation of the **logical relationship** existing between individual elements of data.
- In other words, a data structure is a way of **organizing** all data items that considers not only the elements stored but also their relationship to each other.

Introduction

- Data structure affects the design of both structural & functional aspects of a program.
- **Program=algorithm + Data Structure**
- You know that a algorithm is a **step by step procedure** to solve a particular function.

Introduction

- That means, algorithm is a set of instruction written to carry out certain tasks & the data structure is the way of organizing the data with their logical relationship retained.
- To develop a program of an algorithm, we should select an **appropriate data structure** for that algorithm.
- Therefore algorithm and its associated data structures from a program.

Basic concept of Data Structure

Basic concept of Data Structure

- Data structure helps you to understand how data is organized and how data flow is managed to **increase efficiency** of any process or program.
- Data structure is the structural representation of **logical relationship** between data elements.
- This means that a data structure **organizes** data items based on the relationship between the data elements.
- **Example:**
 - A house can be identified by the house name, location, number of floors and so on.
 - These structured set of variables depend on each other to identify the exact house.
 - Similarly, data structure is a structured set of variables that are linked to each other, which forms the basic component of a system

Basic concept of Data Structure

Any course on Data Structures and Algorithms will try to teach you about three things:

1. It will present a collection of commonly used **data structures and algorithms**. These form a programmer's basic "toolkit". For many problems, some data structure or algorithm in the toolkit will provide a good solution. We focus on data structures and algorithms that have proven over time to be most useful.
2. It will introduce the idea of tradeoffs, and reinforce the concept that there are **costs and benefits** associated with every data structure or algorithm. This is done by describing, for each data structure, the amount of **space and time** required for typical operations. For each algorithm, we examine the time required for key input types.

Basic concept of Data Structure

3. It will teach you how to measure the **effectiveness** of a data structure or algorithm. Only through such measurement can you determine which data structure in your toolkit is most appropriate for a new problem. The techniques presented also allow you to judge the merits of new data structures that you or others might invent.

There are often many approaches to solving a problem. How do we choose between them? At the heart of computer program design are two (sometimes conflicting) goals:

1. To design an algorithm that is easy to understand, code, and debug.
2. To design an algorithm that makes efficient use of the computer's resources.

Basic Terminology

Basic Terminology

- Data structures are the building blocks of any program or the software.
- Choosing the appropriate data structure for a program is the most difficult task for a programmer.
- Following terminology is used as far as data structures are concerned

Data:

- Data can be defined as an **elementary value** or the **collection of values**,
- For example, student's name and its id are the data about the student.

Group Items:

- Data items which have **subordinate data items** are called Group item,
- For example, name of a student can have first name and the last name.

Basic Terminology

Record:

- Record can be defined as the **collection of various data items**,
- for example, if we talk about the student entity, then its name, address, course and marks can be grouped together to form the record for the student.

File:

- A File is a **collection of various records** of one type of entity,
- for example, if there are 60 students in the class, then there will be 20 records in the related file where each record contains the data about each student.

Basic Terminology

Attribute and Entity:

- An entity represents the **class of certain objects**.
- It contains various **attributes**.
- Each attribute represents the particular **property of that entity**.

Field:

- Field is a **single elementary unit** of information representing the attribute of an entity.

A Philosophy of Data Structures

A Philosophy of Data Structures

- You might think that with ever more powerful computers, program efficiency is becoming less important.
- After all, **processor speed and memory size** still continue to improve. Won't today's efficiency problem be solved by tomorrow's hardware?
- As we develop more powerful computers, our history so far has always been to use that additional computing power to tackle more complex problems, be it in the form of more sophisticated user interfaces, bigger problem sizes, or new problems previously deemed computationally infeasible.
- More **complex problems demand more computation**, making the need for efficient programs even greater.

A Philosophy of Data Structures

- Unfortunately, as tasks become more complex, they become less like our everyday experience. So today's computer scientists must be trained to have a thorough understanding of the principles behind **efficient program design**, because their ordinary life experiences often do not apply when designing computer programs.
- In the most general sense, a **data structure** is any data representation and its associated operations.
- Even an **integer or floating point number** stored on the computer can be viewed as a simple data structure.
- More commonly, people use the term “data structure” to mean an **organization or structuring for a collection of data items**.

A Philosophy of Data Structures

- A sorted list of integers stored in an array is an example of such a **structuring**. These ideas are explored further in a discussion of **Abstract Data Types**.
- Given sufficient **space to store a collection of data items**, it is always possible to **search for specified items** within the collection, print or otherwise process the data items in any desired order, or modify the value of any particular data item.
- The most obvious example is an **unsorted array** containing all of the data items.
- It is possible to perform all necessary operations on an unsorted array.
- However, using the proper data structure can make the difference between a program running in a few seconds and one requiring many days.

A Philosophy of Data Structures

- For example, **searching** for a given record in a **hash table** is much faster than searching for it in an unsorted array.
- A solution is said to be **efficient** if it solves the problem within the required **resource constraints**.
- Examples of resource constraints include the total space available to store the data—possibly divided into separate main memory and disk space constraints—and the time allowed to perform each subtask.
- A solution is sometimes said to be efficient if it requires fewer resources than known alternatives, regardless of whether it meets any particular requirements.

A Philosophy of Data Structures

- The **cost** of a solution is the amount of resources that the solution consumes.
- Most often, **cost** is measured in terms of one key resource such as time, with the implied assumption that the solution meets the other resource constraints.

Selecting a Data Structure

Selecting a Data Structure

- It should go without saying that people **write programs to solve problems**. However, sometimes programmers forget this.
- So it is crucial to keep this truism in mind when **selecting a data structure** to solve a particular **problem**.
- Only by **first analyzing the problem to determine the performance goals** that must be achieved can there be any hope of selecting the right data structure for the job.
- Poor program designers ignore this analysis step and apply a data structure that they are familiar with but which is **inappropriate** to the problem. The result is typically a **slow program**.
- Conversely, there is no sense in adopting a complex representation to **“improve”** a program that can meet its performance goals when implemented using a simpler design.

Selecting a Data Structure

- When selecting a data structure to solve a problem, you should follow these steps.
 - Analyze your problem to determine the **basic operations** that must be supported. Examples of basic operations include **inserting** a data item into the data structure, **deleting** a data item from the data structure, and **finding** a specified data item.
 - Quantify the **resource constraints** for each operation.
 - **Select the data structure** that best meets these requirements.
- This three-step approach to selecting a data structure operationalizes a data-centered view of the design process.
- The first concern is for the **data and the operations** to be performed on them, the next concern is the **representation** for those data, and the final concern is the **implementation** of that representation.

Selecting a Data Structure

- **Resource constraints** on certain key operations, such as **search**, **inserting** data records, and **deleting** data records, normally drive the data structure selection process.
- Many issues relating to the relative importance of these operations are addressed by the **following three questions**, which you should ask yourself whenever you must choose a data structure.
 1. **Are all data items inserted into the data structure at the beginning, or are insertions interspersed with other operations? Static applications** (where the data are loaded at the beginning and never change) typically get by with simpler data structures to get an efficient implementation, while **dynamic applications** often require something more complicated.

Selecting a Data Structure

2. **Can data items be deleted?** If so, this will probably make the implementation more complicated.
 3. **Are all data items processed in some well-defined order, or is search for specific data items allowed?** “Random access” search generally requires more complex data structures.
- Each data structure has associated **costs and benefits**. In practice, it is hardly ever true that one data structure is better than another for **use in all situations**.
 - If one data structure or algorithm is superior to another in all respects, the inferior one will usually have long been forgotten.
 - A data structure requires a certain **amount of space** for each data item it stores, a certain **amount of time** to perform a single basic operation, and a certain **amount of programming** effort. **Each problem has constraints on available space and time.**

Selecting a Data Structure

- **Each solution to a problem makes use of the basic operations** in some relative proportion, and the data structure selection process must account for this.
- Only after a careful analysis of your **problem's characteristics** can you **determine the best data structure** for the task.

Example 1

- A bank must support many types of transactions with its customers, but we will examine a simple model where customers wish to open accounts, close accounts, and add money or withdraw money from accounts. We can consider this problem at two distinct levels:
 1. the requirements for the **physical infrastructure and workflow process** that the bank uses in its interactions with its customers, and
 2. the requirements for the **database system** that manages the accounts.
- The typical customer opens and closes accounts far less often than accessing the account. Customers are willing to spend many minutes during the process of opening or closing the account, but are typically not willing to wait more than a brief time for individual account transactions such as a deposit or withdrawal.
- These observations can be considered as informal specifications for the **time constraints on the problem**.

Example 1

- It is common practice for banks to provide **two tiers of service**. Human tellers or automated teller machines (ATMs) support customer access to account balances and updates such as deposits and withdrawals.
- Special service representatives are typically provided (during restricted hours) to handle opening and closing accounts. Teller and ATM transactions are expected to take little time.
- Opening or closing an account can take much longer (perhaps up to an hour from the customer's perspective).
- From a database perspective, we see that ATM transactions do not modify the database significantly.
- For simplicity, assume that if money is added or removed, this transaction simply changes the value stored in an account record.
- Adding a new account to the database is allowed to take several minutes.
- Deleting an account need have no time constraint, because from the customer's point of view all that matters is that all the money be returned (equivalent to a withdrawal).
- From the bank's point of view, the account record might be removed from the database system after business hours, or at the end of the monthly account cycle.

Example 1

- When considering the choice of data structure to use in the database system that manages customer accounts, we see that a data structure that has little concern for the cost of deletion, but is highly efficient for search and moderately efficient for insertion, should meet the resource constraints imposed by this problem.
- Records are accessible by **unique account number** (sometimes called an exact-match query).
- One data structure that meets these requirements is the hash table.
- Hash tables allow for extremely **fast exact-match search**.
- A record can be modified quickly when the modification does not affect its space requirements.
- Hash tables also support efficient insertion of new records. While deletions can also be supported efficiently, too many deletions lead to some degradation in performance for the remaining operations.
- However, the hash table can be reorganized periodically to restore the system to peak efficiency. Such reorganization can occur offline so as not to affect ATM transactions.

Example 2

- A company is developing a **database system containing information about cities and towns** in the United States.
- There are many thousands of cities and towns, and the database program should allow users to find information about a particular place by name (another example of an exact-match query).
- Users should also be able to find all places that match a particular value or range of values for attributes such as location or population size. This is known as a range query.
- A reasonable database system must answer queries quickly enough to satisfy the patience of a typical user.
- For an exact-match query, a few seconds is satisfactory. If the database is meant to support range queries that can return many cities that match the query specification, the user might tolerate the entire operation to take longer, perhaps on the order of a minute.
- To meet this requirement, it will be necessary to support operations that process range queries efficiently by processing all cities in the range as a batch, rather than as a series of operations on individual cities.

Example 2

- The hash table suggested in the previous example is inappropriate for implementing our city database, because it cannot perform efficient range queries.
- The B₊-tree supports large databases, insertion and deletion of data records, and range queries.
- However, a simple linear index would be more appropriate if the database is created once, and then never changed, such as an atlas distributed on a CD or accessed from a website.

Need for Data Structure

Need for Data Structure

- It gives different level of organization data.
- It tells how data can be stored and accessed in its elementary level.
- Provide operation on group of data, such as adding an item, looking up highest priority item.
- Provide a means to manage huge amount of data efficiently.
- Provide fast searching and sorting of data.

Goals of Data Structure

Goals of Data Structure

- Data structure basically implements two complementary goals.

Correctness:

- Data structure is designed such that it operates **correctly** for all kinds of input, which is based on the domain of interest.
- In other words, correctness forms the primary goal of data structure, which always depends on the specific problems that the data structure is intended to solve.

Efficiency:

- Data structure also needs to be **efficient**. It should process the data at high speed without utilizing much of the computer resources such as **memory space**.
- In a real time state, the **efficiency** of a data structure is an important factor that determines the **success and failure of the process**.

Features of Data Structure

Features of Data Structure

- Some of the important features of data structures are:

Robustness:

- Generally, all computer programmers wish to produce software that generates correct output for every possible input provided to it, as well as execute **efficiently** on all hardware platforms.
- This kind of **robust software** must be able to manage both valid and invalid inputs.

Adaptability:

- Developing software projects such as word processors, Web browsers and Internet search engine involves large software systems that work or execute correctly and efficiently for many years.
- Moreover, software evolves due to ever changing market conditions or due to emerging technologies.

Features of Data Structure

Reusability:

- **Reusability and adaptability** go hand-in-hand.
- It is a known fact that the programmer requires many resources for developing any software, which makes it an expensive enterprise.
- However, if the software is developed in a **reusable and adaptable** way, then it can be implemented in most of the future applications.
- Thus, by implementing quality data structures, it is possible to develop reusable software, which tends to be **cost effective and time saving**.

Cost and Benefits

Cost and Benefits

- Each data structure has **costs and benefits**.
- Rarely is one data structure better than another in all situations.
- **Any data structure requires:**
 - space for each data item it stores
 - time to perform each basic operations
 - programming effort.
- Each problem has constraints on available space and time
- Only after a careful analysis of **problem characteristics** we can know the **best data structure for the task**

Cost and Benefits

- Bank example:
 - Start account: a few minutes
 - Transactions: a few seconds
 - Close account: overnight

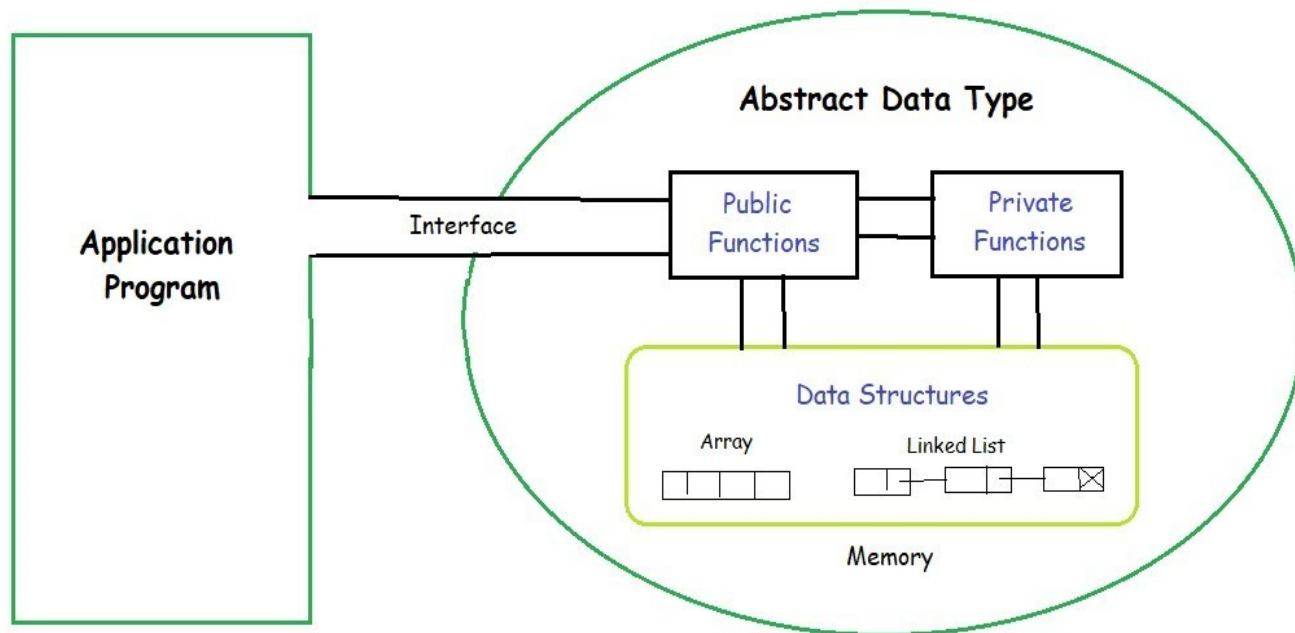
Example 1

- **Problem:** Create a database containing information about cities and towns.
- **Tasks:** Find by name or attribute or location
- Exact match, range query, spatial query
- **Resource requirements:** Times can be from a few seconds for simple queries to a minute or two for complex queries

ABSTRACT DATA TYPE

ABSTRACT DATA TYPE

- According to National Institute of Standards and Technology (NIST), a data structure is an organization of information, usually in the memory, for better algorithm efficiency.



ABSTRACT DATA TYPE

- Data structures include **queues, stacks, linked lists, dictionary, and trees.**
- They could also be a **conceptual entity**, such as the name and address of a person.
- From the above definition, it is clear that the operations in data structure involve **higher-level abstractions** such as, **adding or deleting** an item from a list, **accessing** the highest priority item in a list, or **searching and sorting** an item in a list.
- When the data structure does such operations, it is called an **abstract data type**.

ABSTRACT DATA TYPE

- It can be defined as a collection of data items together with the operations on the data.
- The word “**abstract**” refers to the **fact that the data and the basic operations defined** on it are being studied independently of how they are implemented.
- It involves what can be done with the data, not how has to be done.

ABSTRACT DATA TYPE

- An implementation of ADT consists of **storage structures** to store the data items and **algorithms** for basic operation.
- All the data structures i.e. **array**, **linked list**, **stack**, **queue** etc are examples of ADT.

1. List ADT

10	20	30	40	50	60
----	----	----	----	----	----

- The data is generally stored in **key sequence** in a list which has a head structure consisting of **count**, **pointers** and **address** of compare function needed to compare the data in the list. The data node contains the **pointer to a data structure** and a **self-referential pointer** which points to the next node in the list.

The List ADT Functions is given below:

- `get()` – Return an element from the list at any given position.
- `insert()` – Insert an element at any position of the list.
- `remove()` – Remove the first occurrence of any element from a non-empty list.
- `removeAt()` – Remove the element at a specified location from a non-empty list.
- `replace()` – Replace an element at any position by another element.
- `size()` – Return the number of elements in the list.
- `isEmpty()` – Return true if the list is empty, otherwise return false.
- `isFull()` – Return true if the list is full, otherwise return false.

2. Stack ADT

- In Stack ADT Implementation instead of data being stored in each node, the pointer to data is stored. The program allocates memory for the **data and address** is passed to the stack ADT. The **head node** and the data nodes are encapsulated in the ADT. The calling function can only see the pointer to the stack. The stack head structure also contains a pointer to top and count of number of entries currently in stack.

50
40
30
20
10

- push() – Insert an element at one end of the stack called top.
- pop() – Remove and return the element at the top of the stack, if it is not empty.
- peek() – Return the element at the top of the stack without removing it, if the stack is not empty.
- size() – Return the number of elements in the stack.
- isEmpty() – Return true if the stack is empty, otherwise return false.
- isFull() – Return true if the stack is full, otherwise return false.

3. Queue ADT

10	20	30	40	50	60
----	----	----	----	----	----

- The queue abstract data type (ADT) follows the basic design of the stack abstract data type. Each node contains a void pointer to the data and the **link pointer to the next element** in the queue. The program's responsibility is to allocate memory for storing the data.
 - enqueue() – Insert an element at the end of the queue.
 - dequeue() – Remove and return the first element of the queue, if the queue is not empty.
 - peek() – Return the element of the queue without removing it, if the queue is not empty.
 - size() – Return the number of elements in the queue.
 - isEmpty() – Return true if the queue is empty, otherwise return false.
 - isFull() – Return true if the queue is full, otherwise return false.

Features of ADT

- Abstract data types (ADTs) are a way of **encapsulating data and operations** on that data into a single unit. Some of the key features of ADTs include:
 - **Abstraction:** The user **does not need to know** the implementation of the data structure only essentials are provided.
 - **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
 - **Robust:** The program is robust and has the **ability to catch errors**.
 - **Encapsulation:** ADTs **hide the internal details** of the data and provide a public interface for users to interact with the data. This allows for easier **maintenance and modification** of the data structure.

Features of ADT

- **Data Abstraction:** ADTs provide a level of **abstraction** from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.
- **Data Structure Independence:** ADTs can be implemented using different data structures, such as arrays or linked lists, **without affecting the functionality** of the ADT.
- **Information Hiding:** ADTs can **protect the integrity** of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity:** ADTs can be combined with other ADTs to form larger, **more complex data structures**. This allows for greater flexibility and modularity in programming.

Overall, ADTs provide a **powerful tool for organizing and manipulating data** in a structured and **efficient manner**.

Advantage of using ADTs

- **Encapsulation:** ADTs provide a way to **encapsulate data and operations** into a single unit, making it easier to manage and modify the data structure.
- **Abstraction:** ADTs allow users to work with data structures without having to know the implementation details, which can **simplify programming and reduce errors**.
- **Data Structure Independence:** ADTs can be implemented using different data structures, which can make it **easier to adapt to changing needs and requirements**.
- **Information Hiding:** ADTs can **protect the integrity of data** by controlling access and preventing unauthorized modifications.
- **Modularity:** ADTs can be combined with other ADTs to form more complex data structures, which can **increase flexibility and modularity** in programming.

Disadvantage of using ADTs

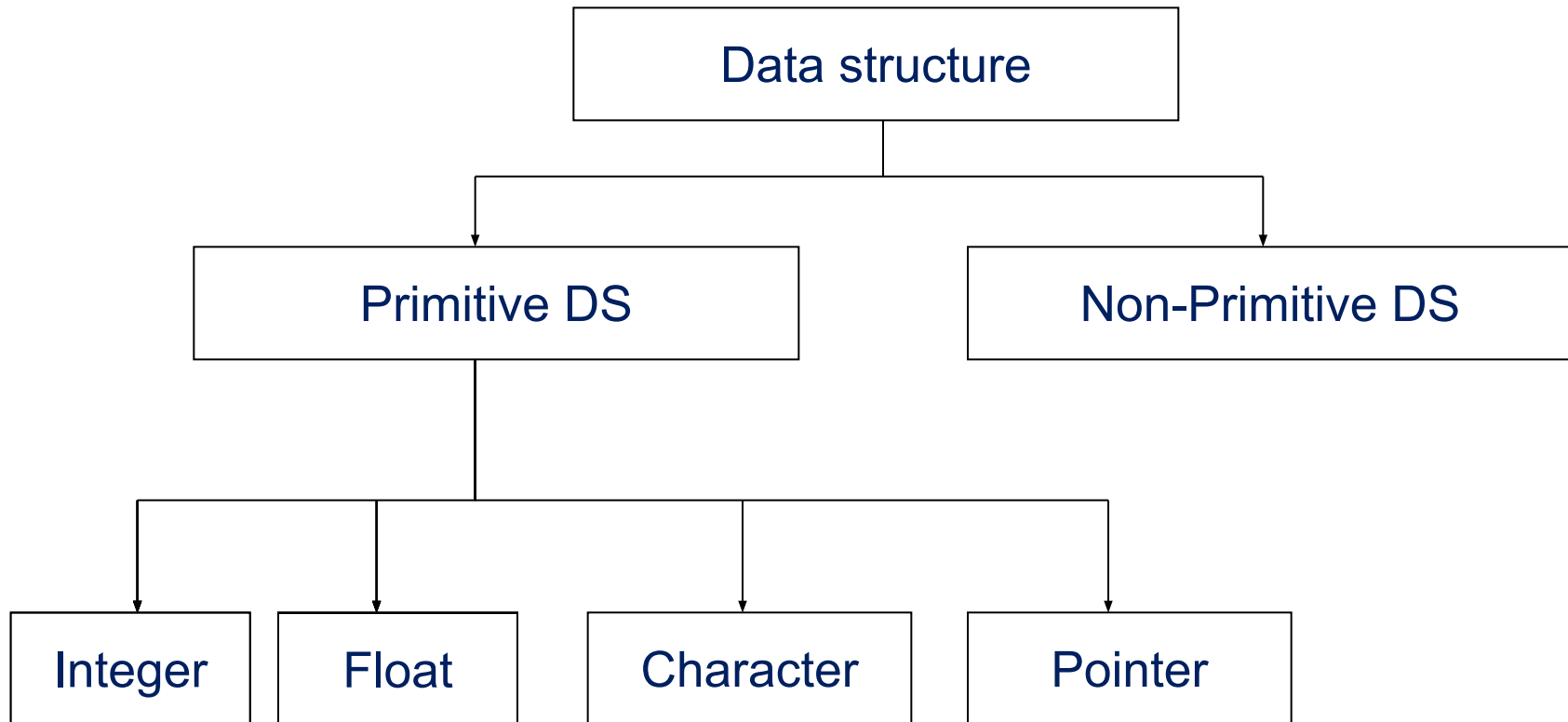
- **Overhead:** Implementing ADTs can add overhead in **terms of memory and processing**, which can affect performance.
- **Complexity:** ADTs can be **complex to implement**, especially for **large and complex data structures**.
- **Learning Curve:** Using ADTs requires knowledge of their implementation and usage, which can **take time and effort** to learn.
- **Limited Flexibility:** Some ADTs may be **limited in their functionality** or may not be suitable for all types of data structures.
- **Cost:** Implementing ADTs may require **additional resources and investment**, which can increase the cost of development.

Classification of Data Structure

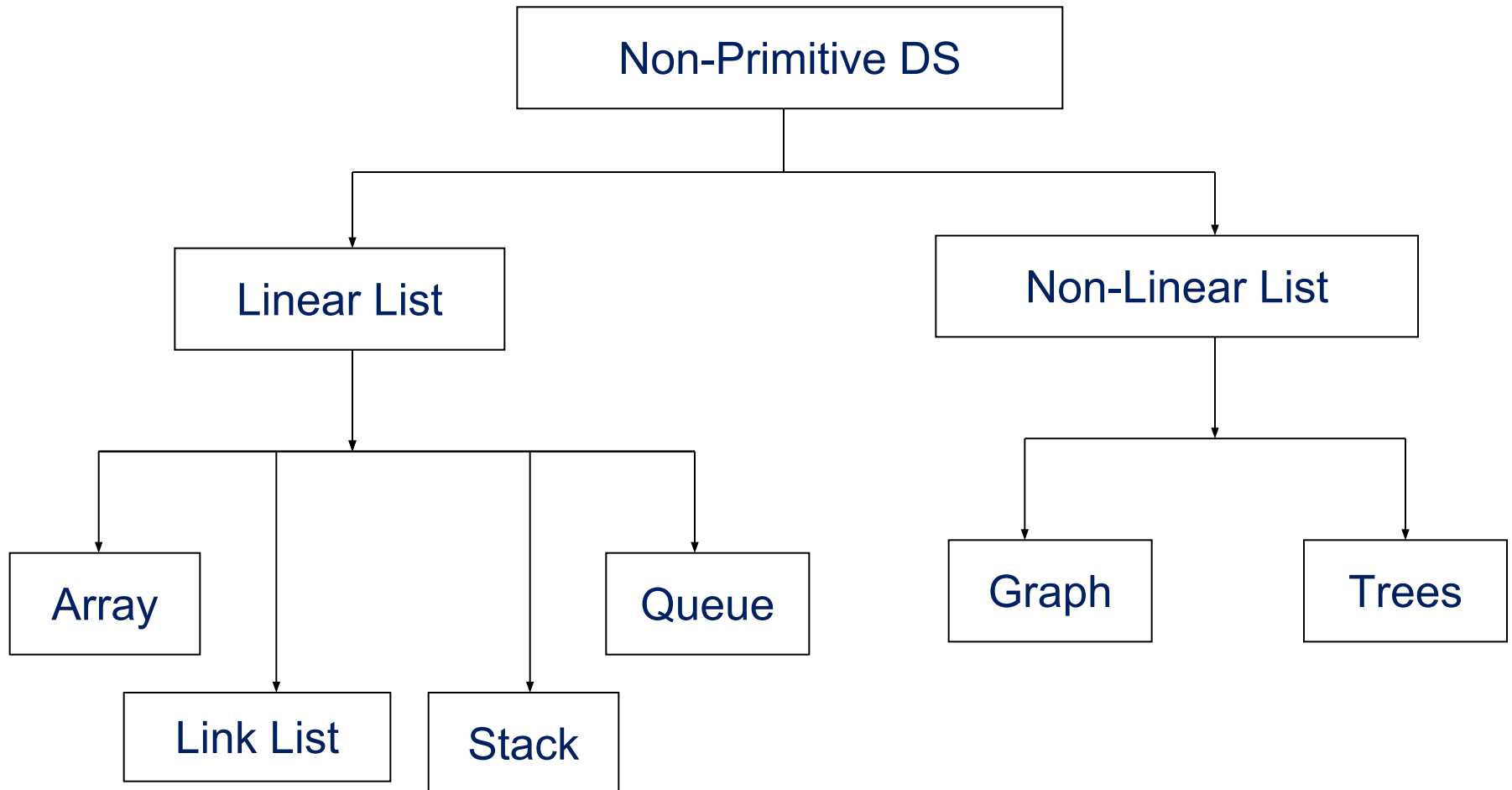
Classification of Data Structure

- Data structure are normally divided into **two broad categories**:
 - Primitive Data Structure
 - Non-Primitive Data Structure

Classification of Data Structure



Classification of Data Structure



Primitive Data Structure

- There are **basic structures** and directly operated upon by the machine instructions.
- In general, there are **different representation** on different computers.
- **Integer, Floating-point number, Character constants, string constants, pointers** etc, fall in this category.

Non-Primitive Data Structure

- There are more **sophisticated data structures**.
- These are **derived from the primitive data structures**.
- The non-primitive data structures emphasize on **structuring of a group of homogeneous (same type) or heterogeneous (different type) data items**.

Non-Primitive Data Structure

- **Lists, Stack, Queue, Tree, Graph** are example of non-primitive data structures.
- The design of an **efficient data structure** must take operations to be performed on the data structure.

Non-Primitive Data Structure

- The most commonly used **operation on data structure** are broadly categorized into following types:
 - Create
 - Selection
 - Updating
 - Searching
 - Sorting
 - Merging
 - Destroy or Delete

Different between them

- A primitive data structure is generally a **basic structure** that is usually built into the language, such as an **integer, a float**.
- A non-primitive data structure is **built out of primitive data structures** linked together in meaningful ways, such as a or a **linked-list, binary search tree, AVL Tree, graph** etc.

STATIC DATA STRUCTURE VS DYNAMIC DATA STRUCTURE

STATIC DATA STRUCTURE VS DYNAMIC DATA STRUCTURE

- Data structure is a way of **storing and organising data efficiently** such that the required operations on them can be performed be efficient with respect to **time** as well as **memory**.
- Simply, Data Structure are used to **reduce complexity** (mostly the time complexity) of the code.
- Data structures can be two types:
 1. Static Data Structure
 2. Dynamic Data Structure

What is a Static Data structure?

- In Static data structure the **size of the structure is fixed**.
- The content of the data structure can be modified but **without changing the memory space allocated** to it.
- Example of Static Data Structures: Array

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

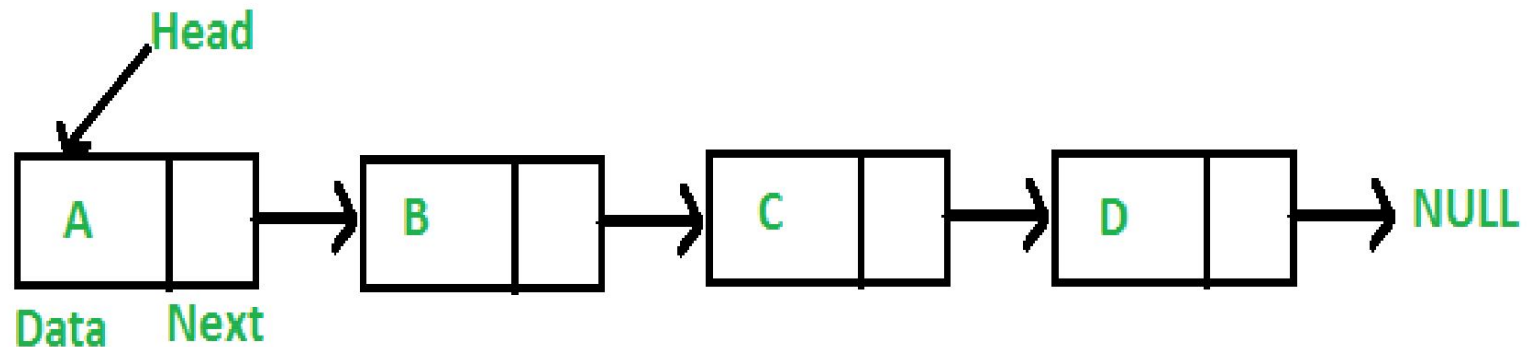
Array Length = 9

First Index = 0

Last Index = 8

What is a Dynamic Data structure?

- In Dynamic data structure the size of the structure is not fixed and **can be modified during the operations** performed on it.
- Dynamic data structures are designed to facilitate **change of data structures in the run time.**
- Example of Dynamic Data Structures: Linked List



Static Data Structure vs Dynamic Data Structure

- Static Data structure has **fixed memory size** whereas in Dynamic Data Structure, the size can **be randomly updated during run time** which may be considered efficient with respect to **memory complexity** of the code.
- Static Data Structure provides more **easier access** to elements with respect to dynamic data structure.
- Unlike static data structures, dynamic data structures are **flexible**.

OPERATIONS ON DATA STRUCTURES

OPERATIONS ON DATA STRUCTURES

- **Traversing** It means to access each data item exactly once so that it can be processed. For example, to print the names of all the students in a class.
- **Searching** It is used to find the location of one or more data items that satisfy the given constraint. Such a data item may or may not be present in the given collection of data items. For example, to find the names of all the students who secured 100 marks in mathematics.
- **Inserting** It is used to add new data items to the given list of data items. For example, to add the details of a new student who has recently joined the course.

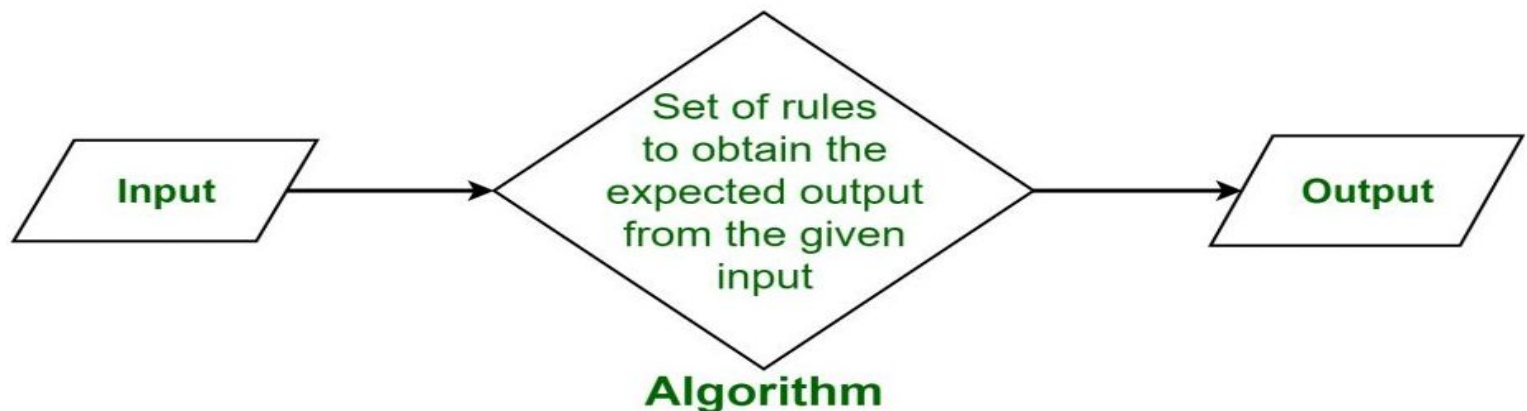
OPERATIONS ON DATA STRUCTURES

- **Deleting** It means to remove (delete) a particular data item from the given collection of data items. For example, to delete the name of a student who has left the course.
- **Sorting** Data items can be arranged in some order like ascending order or descending order depending on the type of application. For example, arranging the names of students in a class in an alphabetical order, or calculating the top three winners by arranging the participants' scores in descending order and then extracting the top three.
- **Merging** Lists of two sorted data items can be combined to form a single list of sorted data items.

ALGORITHM

ALGORITHM

- Algorithm is a **step-by-step procedure**, which defines a **set of instructions** to be executed
- in a **certain order** to get the desired output.
- Algorithms are generally created **independent** of underlying languages, i.e. an algorithm can be implemented in more than one programming language.



Algorithm + Data structure = Program

ALGORITHM

- From the data structure point of view, following are some important categories of algorithms –
 - **Search** – Algorithm to search an item in a data structure.
 - **Sort** – Algorithm to sort items in a certain order.
 - **Insert** – Algorithm to insert item in a data structure.
 - **Update** – Algorithm to update an existing item in a data structure.
 - **Delete** – Algorithm to delete an existing item from a data structure.

Characteristics of an Algorithm

An algorithm should have the following characteristics –

- Clear and Unambiguous:** Algorithm should be **clear and unambiguous**. Each of its steps should be clear in all aspects and must lead to only one meaning.
- Well-Defined Inputs:** If an algorithm says to take inputs, it should be **welldefined** inputs.
- Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be **well-defined** as well.

Characteristics of an Algorithm

- **Finite-ness:** The algorithm must be **finite**, i.e. it should not end up in an infinite loops or similar.
- **Feasible:** The algorithm must be **simple, generic and practical**, such that it can be executed upon will the available resources. It must not contain some future technology, or anything.
- **Language Independent:** The Algorithm designed must be **language independent**, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be same, as expected.

Advantages and Disadvantages of Algorithm

Advantages of Algorithms:

- It is **easy** to understand.
- Algorithm is a **step-wise** representation of a solution to a given problem.
- In Algorithm the problem is **broken down into smaller pieces** or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- Writing an algorithm takes a long time so it is **time-consuming**.
- **Branching and Looping** statements are difficult to show in Algorithms.

Different approach to design an algorithm

1. **Top-Down Approach:** A top-down approach **starts with identifying major components** of system or program decomposing them **into their lower level components** & iterating until desired level of module complexity is achieved. In this we start with topmost module & incrementally add modules that is calls.

2. **Bottom-Up Approach:** A bottom-up approach starts with designing most basic or primitive component & proceeds **to higher level components**. Starting from very bottom, operations that provide layer of abstraction are implemented

How to Write an Algorithm?

- There are no well-defined standards for writing algorithms.
- Rather, it is **problem and resource** dependent.
- Algorithms are never written to support a particular programming code.
- As we know that all programming languages share basic code constructs like loops (do, for, while), flow-control (if-else), etc.
- These common constructs can be used to write an algorithm.

How to Write an Algorithm?

- We write algorithms in a **step-by-step manner**, but it is not always the case.
- Algorithm writing is a process and is executed after the problem domain is **well-defined**.
- That is, we should know the problem domain, for which we are designing a solution.

How to Write an Algorithm?

Example

- Let's try to learn algorithm-writing by using an example.
- Problem – Design an algorithm to add two numbers and display the result.

Method-1

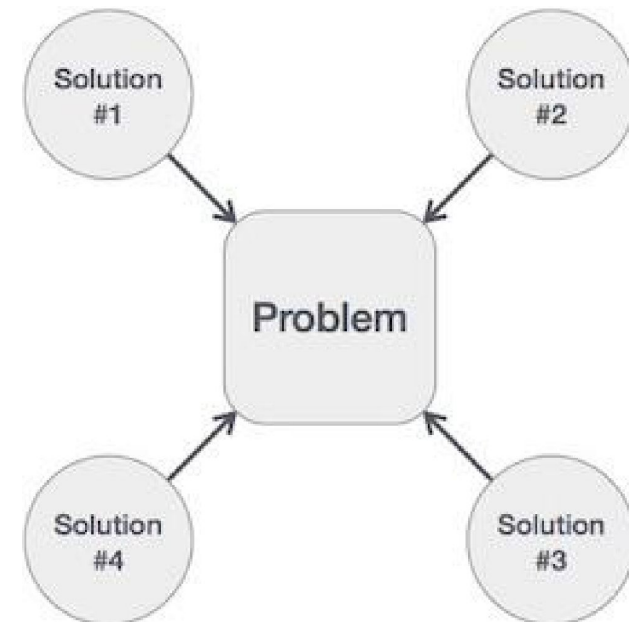
Step 1 – START
Step 2 – declare three integers a, b & c
Step 3 – define values of a & b
Step 4 – add values of a & b
Step 5 – store output of step 4 to c
Step 6 – print c
Step 7 – STOP

Method-2

Step 1 – START ADD
Step 2 – get values of a & b
Step 3 – $c \leftarrow a + b$
Step 4 – display c
Step 5 – STOP

How to Write an Algorithm?

- In design and analysis of algorithms, usually the second method is used to describe an algorithm.
- It makes it easy for the analyst to analyze the algorithm **ignoring all unwanted definitions.**
- It can observe what operations are being used and how the process is flowing.
- Writing step numbers, is optional.
- We design an algorithm to get a solution of a given problem.
- A problem can be solved in more than one ways.



ALGORITHM COMPLEXITY

- Suppose X is an algorithm and n is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the **efficiency** of X .
 - **Time Factor** – Time is measured by counting the **number of key operations** such as comparisons in the sorting algorithm.
 - **Space Factor** – Space is measured by **counting the maximum memory space** required by the algorithm.
- The complexity of an algorithm $f(n)$ gives the **running time and/or the storage space** required by the algorithm in terms of n as the size of input data.

ALGORITHM COMPLEXITY

1. Space Complexity

- Space complexity of an algorithm represents the **amount of memory space** required by the algorithm in its life cycle.
- The space required by an algorithm is equal to the sum of the following two components –
 - A **fixed part** that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and **constants** used, program size, etc.
 - A **variable part** is a space required by variables, whose size depends on the size of the problem. For example, **dynamic memory allocation, recursion stack space**, etc.

ALGORITHM COMPLEXITY

- Space complexity $S(P)$ of any algorithm P is $S(P) = C + SP(I)$, where C is the fixed part and $S(I)$ is the variable part of the algorithm, which depends on instance characteristic I .
- Following is a simple example that tries to explain the concept **Algorithm: SUM(A, B)**
 - Step 1 - START
 - Step 2 - $C \leftarrow A + B + 10$
 - Step 3 - Stop
- Here we have three variables A , B , and C and one constant. Hence $S(P) = 1 + 3$. Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

ALGORITHM COMPLEXITY

2. Time Complexity

- Time complexity of an algorithm represents the **amount of time required** by the algorithm to run to completion.
- Time requirements can be defined as a numerical function $T(n)$, where $T(n)$ can be measured as the **number of steps**, provided each step consumes constant time.
- For example, addition of two n -bit integers takes n steps. Consequently, the total computational time is $T(n) = c * n$, where c is the time taken for the addition of two bits.
- Here, we observe that $T(n)$ grows linearly as the input size increases.

Program Design

There are various ways by which we can specify an program design. Those are -

1.Use of Pseudocode

2.Representation of Flowchart

1. Pseudocode Definition: Pseudo code is nothing but an informal way of writing a Program. It is a combination of algorithm written in English and some programming language. There is no restriction of following syntax of programming language. Pseudo code cannot be compiled; it is just a previous step of developing a code in algorithm.

Program Design

2. Flowchart - Flowcharts are the graphical representation of the algorithms. The algorithms and flowcharts are the final steps in organizing the solutions. Using the algorithms and flowcharts the programmers can find out the bugs in the programming logic and then can go for coding. Flowcharts can show errors in the logic and set of data can be easily tested using flowcharts.

Program Design

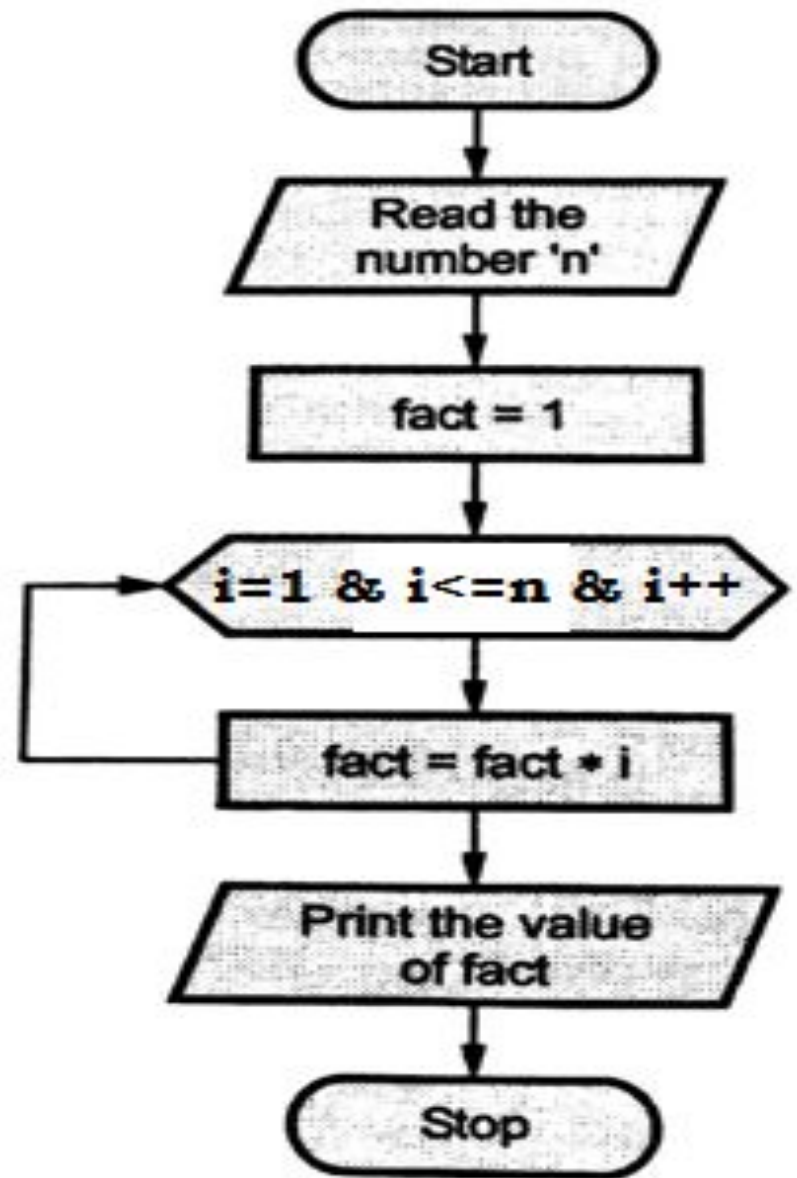
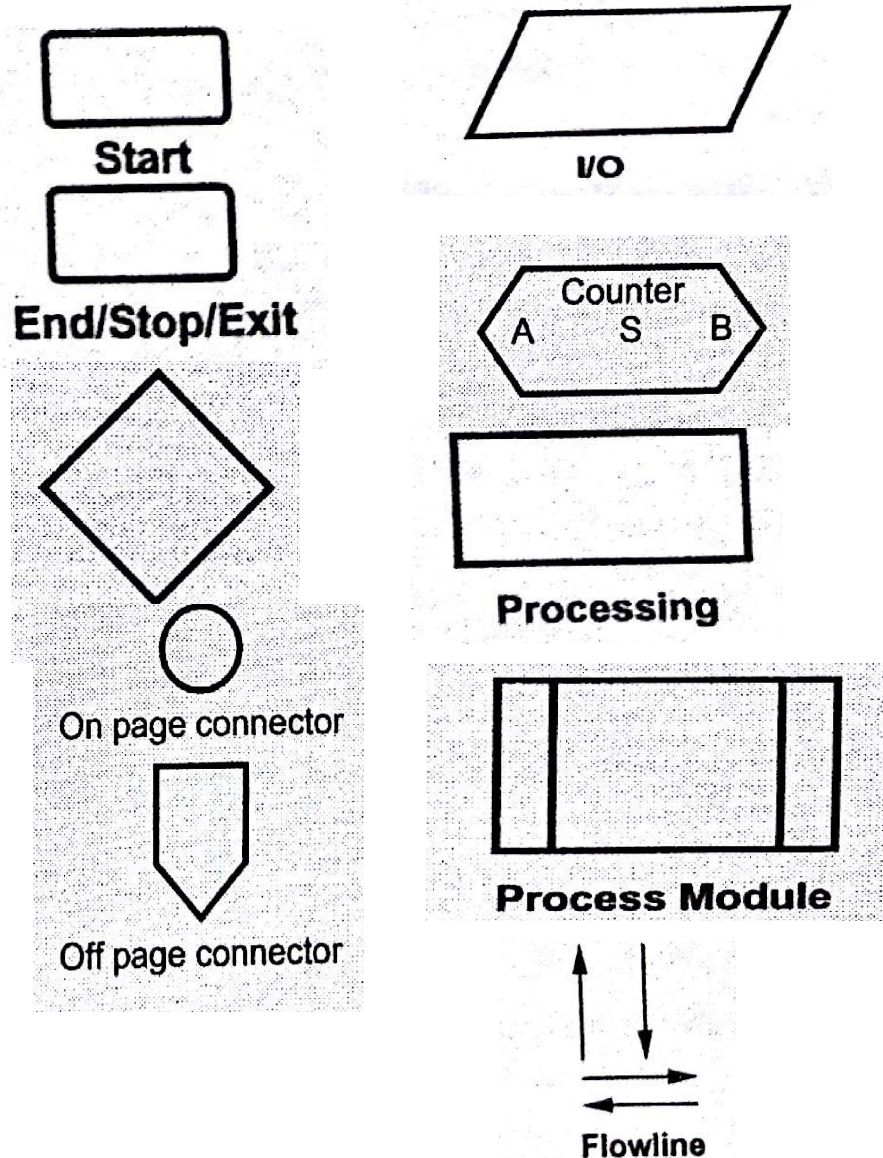


Figure 11 symbol of flowchart

Figure 12 Flowchart of factorial

ALGORITHM ANALYSIS

ALGORITHM ANALYSIS

Efficiency of an algorithm can be analyzed at two different stages, **before implementation and after implementation**.

They are the following –

- **A Priori Analysis or Performance Analysis** – This is a theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that **all other factors**, for example, **processor speed**, are constant and have no effect on the implementation.
- **A Posterior Analysis or Performance Measurement** – This is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual **statistics** like **running time and space required**, are collected.

ALGORITHM ANALYSIS

- Priori algorithm analysis, algorithm analysis deals with the **execution or running time** of various operations involved.
- The running time of an operation can be defined as the **number of computer instructions executed per operation**.
- Analysis of an algorithm is required to determine the **amount of resources** such as **time and storage** necessary to execute the algorithm.
- Usually, the efficiency or running time of an algorithm is stated as a function which relates the **input length to the time complexity or space complexity**.

ALGORITHM ANALYSIS

- Algorithm analysis framework involves finding out the **time taken and the memory space** required by a program to execute the program.
- It also determines how the **input size of a program** influences the **running time** of the program.
- In theoretical analysis of algorithms, it is common to estimate their complexity in the asymptotic sense, i.e., to estimate the complexity function for arbitrarily large input.
- **Big- O notation, Omega notation, and Theta notation** are used to **estimate the complexity function** for large arbitrary input.

Types of Analysis

- The efficiency of some algorithms may **vary for inputs** of the same size.
- For such algorithms, we need to differentiate between the **worst case, average case and best case efficiencies**.

Best Case Analysis

- If an algorithm takes the **least amount of time to execute** a specific set of input, then it is called the **best case time complexity**.
- The best case efficiency of an algorithm is the efficiency for the **best case input of size n** .
- Because of this input, the algorithm **runs the fastest among all the possible inputs of the same size**.

Types of Analysis

Average Case Analysis

- If the time complexity of an algorithm for **certain sets of inputs are on an average**, then such a time complexity is called **average case time complexity**.
- Average case analysis provides necessary information about an **algorithm's behavior** on a typical or random input.
- You must make some assumption about the **possible inputs of size n to analyze the average case** efficiency of algorithm.

Types of Analysis

Worst Case Analysis

- If an algorithm takes **maximum amount of time to execute** for a specific set of input, then it is called the **worst case time complexity**.
- The worst case efficiency of an algorithm is the efficiency for the worst case input of size n .
- The algorithm runs the **longest among all the possible inputs** of the similar size because of this input of size n .

Description of various Data Structures

Description of various Data Structures : Arrays

- Array, in general, refers to an **orderly arrangement** of data elements.
- Array is a type of data structure that stores data elements in **adjacent locations**.
- Array is considered as **linear data structure** that stores elements of **same data types**.
- Hence, it is also called as a **linear homogenous data structure**.

Arrays

- When we declare an array, we can assign initial values to each of its elements by enclosing the values in braces { }.
- **int Num [5] = { 26, 7, 67, 50, 66 };**
- This declaration will create an array as shown below:

	0	1	2	3	4
Num	10	20	30	40	50

Arrays

A program of array to find average marks.

```
main( )
{
    int avg, sum = 0 ; int i ; int marks[30] ; /* array declaration */
    clrscr();
    for ( i = 0 ; i <= 29 ; i++ )
    { printf ( "\nEnter marks " ) ;
      scanf ( "%d", &marks[i] ) ; /* store data in array */
    }
    for ( i = 0 ; i <= 29 ; i++ )
    sum = sum + marks[i] ; /* read data from an array*/
    avg = sum / 30 ; printf ( "\nAverage marks = %d", avg ) ;
    getch();
}
```

Arrays

- Arrays can be classified as one-dimensional array, two-dimensional array or multidimensional array.
- **One-dimensional Array:** It has only one row of elements. It is stored in ascending storage location.
- `int a[10] ;`
- `int num[5] = { 2, 4, 12, 5, 45 } ;`
- `int n[ ] = { 2, 4, 12, 5, 45 } ;`

	0	1	2	3	4
num	2	4	12	5	45

Arrays

- **Two-dimensional Array:** It consists of multiple rows and columns of data elements. It is also called as a matrix.

```
int stud[4][2] = { { 1234, 56 }, { 1212, 33 }, { 1434, 80 }, { 1312, 78 } } ;
```

- A sample program that stores roll number and marks obtained by a student side by side in a matrix.

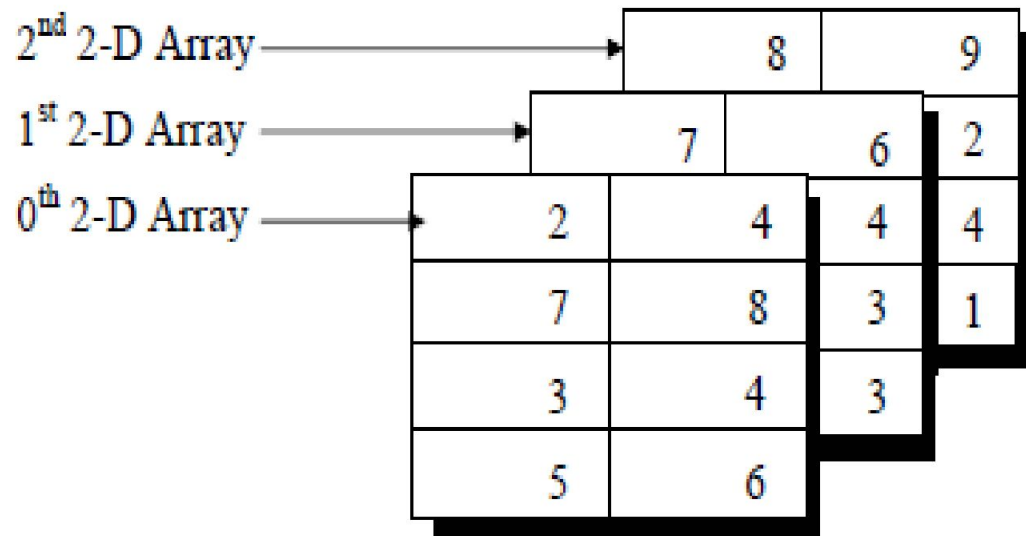
```
main( ) {  
int stud[4][2] ; int i, j ;  
clrscr();  
for ( i = 0 ; i <= 3 ; i++ ) {  
printf ( "\n Enter roll no. and marks" ) ;  
scanf ( "%d %d", &stud[i][0], &stud[i][1] ) ;  
}  
for ( i = 0 ; i <= 3 ; i++ )  
printf ( "\n%d %d", stud[i][0], stud[i][1] ) ;  
getch();  
}
```

	col. no. 0	col. no. 1
row no. 0	1234	56
row no. 1	1212	33
row no. 2	1434	80
row no. 3	1312	78

Arrays

- **Multidimensional Array:** Multidimensional arrays can be defined as **array of arrays**. Multidimensional arrays are not bounded to two indices or two dimensions. They can include as **many indices as required**.

```
int arr[4][2][3] = { { { 2, 4 }, { 7, 8 }, { 3, 4 }, { 5, 6 } }, { { 7, 6 }, { 3, 4 }, { 5, 3 },  
    { 2, 3 } }, { { 8, 9 }, { 7, 2 }, { 3, 4 }, { 5, 1 }, } };
```



Arrays

Advantages of Arrays:

1. Arrays store multiple data of **similar types** with the same name.
2. It allows **random access** to elements.
3. As the array is **of fixed size** and stored **in contiguous memory locations** there is no memory shortage or overflow.
4. It is helpful to store any type of data with a fixed size.
5. Since the elements in the array are stored at **contiguous memory locations** it is easy to iterate in this data structure and unit time is required to access an element if the index is known.

Arrays

Disadvantages of Arrays:

1. The array is **static in nature**. Once the size of the array is declared then we can't modify it.
2. **Insertion and deletion operations are difficult** in an array as elements are stored in contiguous memory locations and the shifting operations are costly.
3. The number of elements that have to be stored in an array should be **known in advance**.
4. **Wastage of memory** is the main problem in the array. If the array size is big the less allocation of memory leads to wastage of memory.

Arrays

Operations:

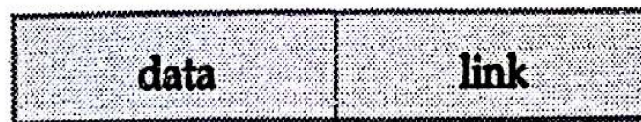
- Insertion of new element
- Deletion of required element
- Modification of an element
- Merging of arrays

Applications:

- Storing list of data elements belonging to same data type
- Auxiliary storage for other data structures
- Storage of binary tree elements of fixed count
- Storage of matrices

Link List Concepts

- A lists (Linear linked list) can be defined as a collection of variable **number of data items**.
- Lists are the most commonly used **non-primitive** data structures.
- An element of list must contain **at least two fields**, one for storing **data or information** and other for storing **address of next element**.
- As you know for storing address we have a special data structure of list the address must be pointer type.

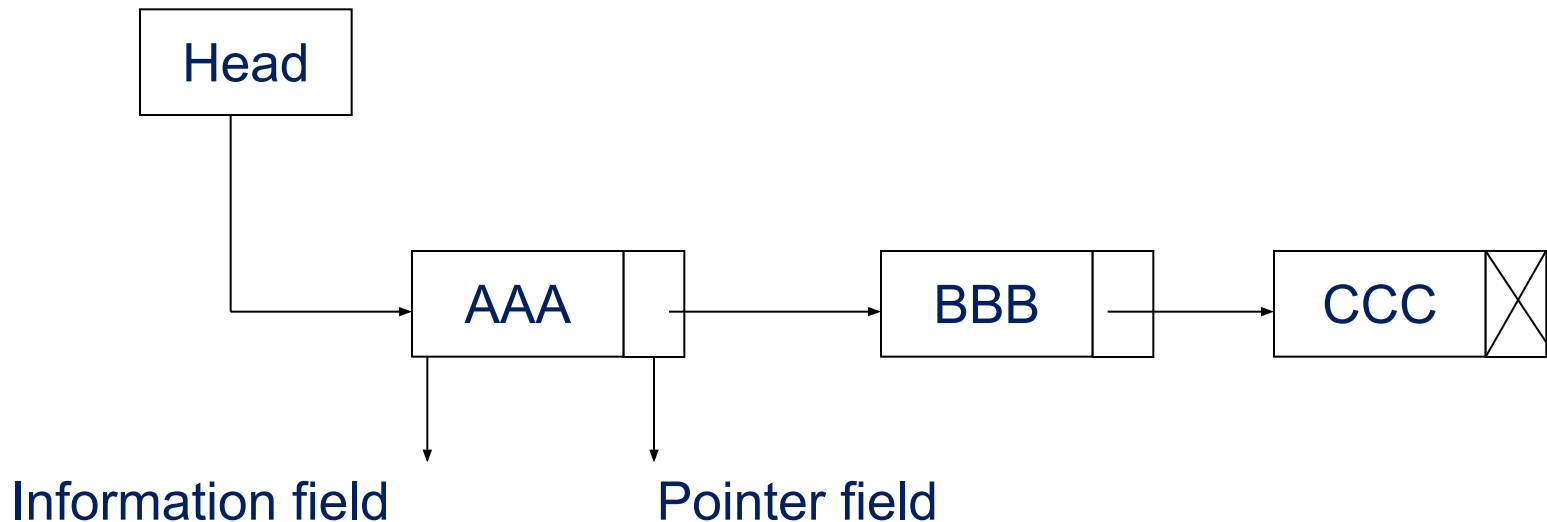


Node of Link List

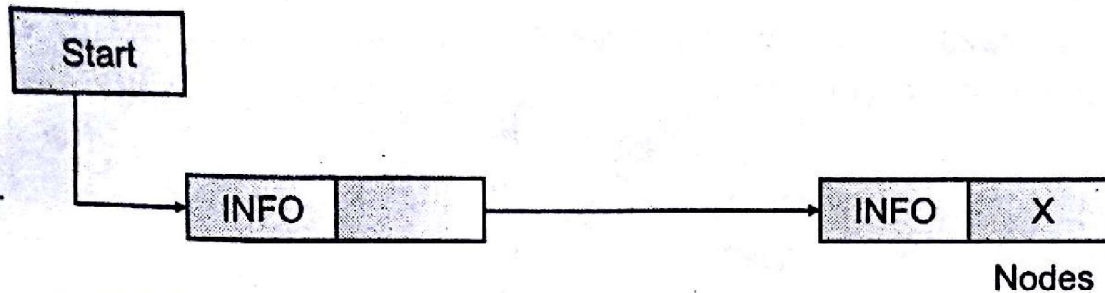
Link List

- Technically each such element is referred to as a node, therefore a list can be defined as a collection of nodes as show bellow:

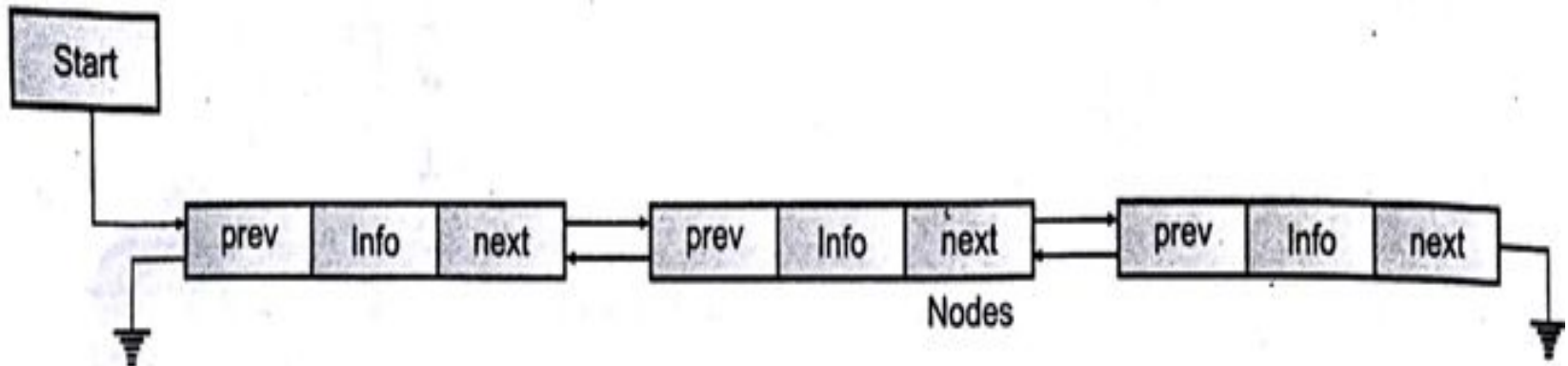
[Linear Liked List]



Link List



(a) Singly Linked Lists



(b) Doubly Linked Lists

Link List

Why linked list data structure needed?

- 1.Dynamic Data structure:** The size of memory can be allocated or de-allocated at run time based on the operation insertion or deletion.
- 2.Ease of Insertion/Deletion:** The insertion and deletion of elements are simpler than arrays since no elements need to be shifted after insertion and deletion, Just the address needed to be updated.
- 3.Efficient Memory Utilization:** As we know Linked List is a dynamic data structure the size increases or decreases as per the requirement so this avoids the wastage of memory.
- 4.Implementation:** Various advanced data structures can be implemented using a linked list like a stack, queue, graph, hash maps, etc.

Link List

Advantages: Linked lists have many advantages. Some of the very important advantages are:

1. Linked lists are **dynamic data structures**. That is, they can grow or shrink during the execution of a program.
2. **Efficient memory utilization**. Here, memory is not pre-allocated. Memory is allocated whenever it is required. And it is deallocated (removed) when it is no longer needed.
3. **Insertion and deletions are easier and efficient**. Linked lists provide flexibility in inserting a data item at a specified position and deletion of a data item from the given position.
4. **Many complex applications** can be easily carried out with linked lists.

Link List

Example: suppose we maintain a sorted list of IDs in an array `id[ ] = [1000, 1010, 1050, 2000, 2040, .....]`. And if we want to insert a new ID 1005, then to maintain the sorted order, we have to move all the elements after 1000 (excluding 1000).

Deletion is also expensive with arrays unless some special techniques are used. For example, to delete 1010 in `id[]`, everything after 1010 has to be moved. So Linked list provides the following two advantages over arrays:

1. Dynamic size
2. Ease of insertion/deletion

Link List

Disadvantages of Linked Lists:

1. **Random access is not allowed.** We have to access elements sequentially starting from the first node. So we cannot do a binary search with linked lists.
2. **Extra memory space** for a pointer is required for each element of the list.
3. Arrays have a better **cache locality** that can make a pretty big difference in performance.
4. It takes a **lot of time in traversing and changing** the pointers.
5. It will be **confusing** when we work with **pointers**.

Link List

Sr. No.	Linked List	Array
1.	The linked list is a collection of nodes and each node is having one data field and next link field. For example <pre> graph LR subgraph Node direction LR Data[Data] --- NextLink[Next link] end </pre>	The array is a collection of similar types of data elements. In arrays the data is always stored at some index of the array. For example <pre> graph TD subgraph Array direction LR a0[10] --- a1[20] --- a2[30] --- a3[40] --- a4[50] end a0 --- i0[a[0]] a1 --- i1[a[1]] a2 --- i2[a[2]] a3 --- i3[a[3]] a4 --- i4[a[4]] Index[Index] --> i1 </pre>
2.	Any element can be accessed by sequential access only.	Any element can be accessed randomly i.e. with the help of index of the array.
3.	Physically the data can be deleted.	Only logical deletion of the data is possible.
4.	Insertions and deletion of data is easy.	Insertions and deletion of data is difficult.
5.	Memory allocation is dynamic. Hence developer can allocate as well as deallocate the memory. And so no wastage of memory is there.	The memory allocation is static. Hence once the fixed amount of size is declared then that much memory is allocated. Therefore there is a chance of either memory wastage or memory shortage.

Link List

Applications:

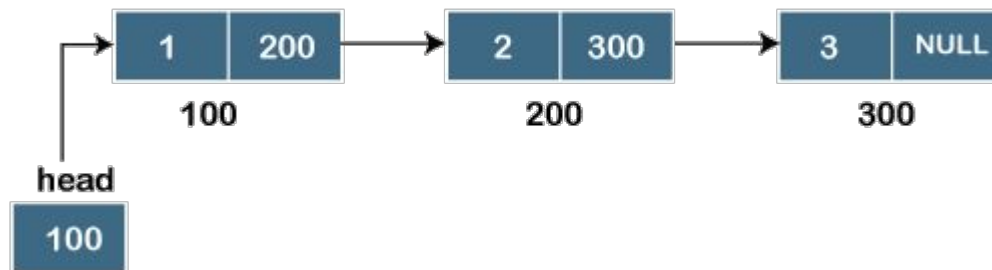
- Implementing **stacks, queues, binary trees and graphs** of predefined size.
- Implement **dynamic memory management** functions of operating system.
- Polynomial implementation for **mathematical operations**
- Circular linked list is used to implement **OS or application functions** that require **round robin execution** of tasks.
- **Circular linked** list is used in a slide show where a user wants to go back to the first slide after last slide is displayed.
- Doubly linked list is used in the implementation **of forward and backward** buttons in a browser to move backwards and forward in the opened pages of a website.
- Circular queue is used to maintain the playing sequence of **multiple players in a game**.

Link List

- Types of linked lists:
 1. Singly linked list
 2. Doubly linked list
 3. Circular linked list
 4. Circular doubly linked list

Singly Link List

- It is the commonly used linked list in programs.
- The singly linked list is a data structure that contains two parts, i.e., **data part**, and **address part**, which contains the address of the next or the successor node.
- The **address part** in a node is also known as a **pointer**.
- Suppose we have three nodes, and the addresses of these three nodes are 100, 200 and 300 respectively.
- The representation of three nodes as a linked list is shown in the below figure:



Singly Link List

- The pointer that holds the address of the initial node is known as a **head pointer**.
- In this list, only **forward traversal is possible**; we cannot traverse in the backward direction as it has only one link in the list.

```
struct node
{
    int data;
    struct node *next;
}
```

- user-defined structure named a **node** containing two members, the first one is data of integer type, and the other one is the pointer (next) of the node type.

Singly Link List

Advantages of the singly linked list are :

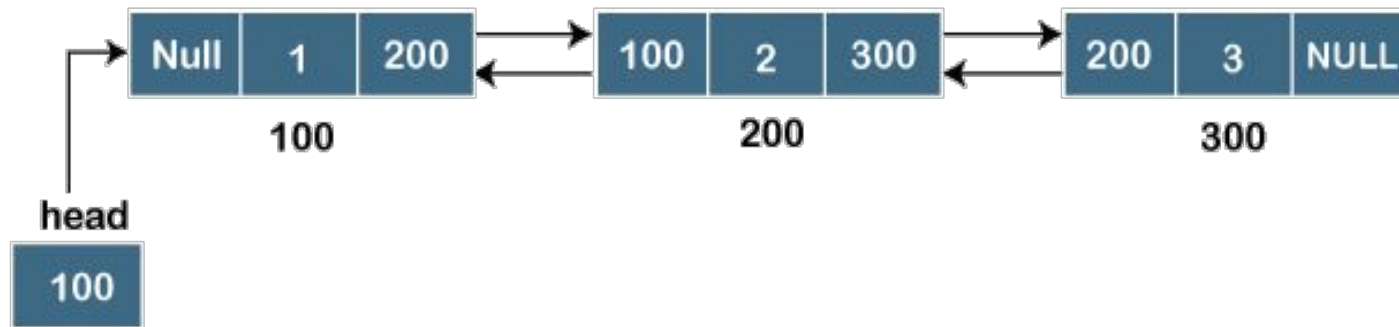
1. Accessibility of a node in the forward direction is easier.
2. Insertion and deletion of nodes are easier.

Disadvantages

1. Accessing the preceding node of a current node is not possible as there is no backward traversal.
2. Accessing a node is time-consuming.

Doubly linked list

- The doubly linked list contains two pointers. We can define the doubly linked list as a linear data structure with three parts: the **data part** and the **other two address part**.
- In other words, a doubly linked list is a list that has three parts in a single node, includes one **data part**, a pointer to its **previous node**, and a pointer to the **next node**.
- Suppose we have three nodes, and the address of these nodes are 100, 200 and 300, respectively. The representation of these nodes in a doubly-linked list is shown below:



Doubly linked list

- Representation of the node in a doubly linked list

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
    struct node *prev;
```

```
}
```

Doubly linked list

Applications of Doubly Linked List:

1. Doubly linked list can be used in **navigation systems** where both forward and backward traversal is required.
2. The doubly linked list is used to implement **data structures** like a stack, queue, binary tree, and hash table.
3. It is also used in algorithms of **LRU** (Least Recently used) and **MRU** (Most Recently Used) cache.
4. It can be used to implement **undo/redo operations**.
5. Doubly linked lists are used in **web page navigation** in both forward and backward directions.
6. It can be used in **games** like a deck of cards.

Doubly linked list

Sr. No.	Singly linked list	Doubly linked list					
1.	Singly linked list is a collection of nodes and each node is having one data field and next link field. For example : <table border="1"><tr><td>Data</td><td>Next link</td></tr></table>	Data	Next link	Doubly linked list is a collection of nodes and each node is having one data field, one previous link field and one next link field. For example : <table border="1"><tr><td>Previous link field</td><td>Data</td><td>Next link field</td></tr></table>	Previous link field	Data	Next link field
Data	Next link						
Previous link field	Data	Next link field					
2.	The elements can be accessed using next link.	The elements can be accessed using both previous link as well as next link.					
3.	No extra field is required; hence node takes less memory in SLL.	One field is required to store previous link hence node takes more memory in DLL.					
4.	Less efficient access to elements.	More efficient access to elements.					

Doubly linked list

Advantages

- 1.Efficient traversal in both forward and backward directions:** With a doubly linked list, you can traverse the list in both forward and backward directions, which can be useful in certain applications.
- 2.Dynamic size adjustment:** The size of the list can be easily adjusted based on the number of elements added or removed, making it a flexible data structure.
- 3.Constant-time insertions and deletions:** Insertions and deletions can be performed in constant time at both the beginning and the end of the list.
- 4.Easy to implement:** The doubly linked list is relatively easy to implement, compared to other data structures like arrays or trees.
- 5.Efficient memory utilization:** The doubly linked list can be used to manage memory efficiently, as nodes can be easily added or removed as needed.

Doubly linked list

Disadvantages

- 1. More memory usage:** Each node in a doubly linked list requires two pointers (previous and next), resulting in higher memory usage compared to a singly linked list.
- 2. Slower access and search times:** Access and search operations have $O(n)$ time complexity, where n is the number of elements in the list. This can result in slower performance compared to other data structures like arrays or trees, especially for large lists.
- 3. Complex implementation:** The implementation of certain operations, such as sorting, can be more complex compared to arrays or other data structures.

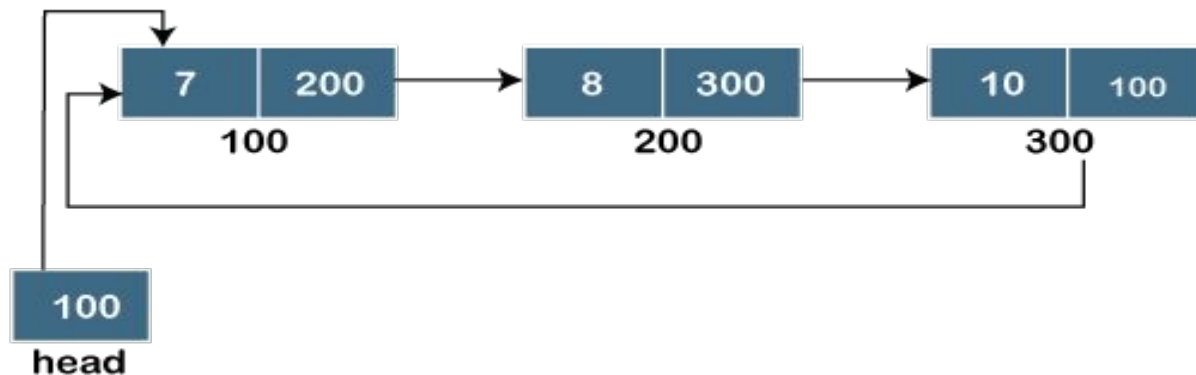
Doubly linked list

Disadvantages

- 4.Higher overhead for updates:** Updates to the list, such as inserting or deleting elements, can be more time-consuming compared to arrays or other data structures.
- 5.Pointer management:** The management of pointers in a doubly linked list can be more complex compared to other data structures, and mistakes in pointer management can result in serious errors in the program.

Circular linked list

- A circular linked list is a variation of a singly linked list.
- The only difference between the singly linked list and a circular linked list is that the last node does not point to any node in a singly linked list, so its link part contains a NULL value.
- On the other hand, the circular linked list is a list in which the **last node connects to the first node**, so the link part of the last node holds the first node's address.
- The circular linked list has **no starting and ending node**.
- We can **traverse in any direction**, i.e., either backward or forward.
The diagrammatic representation of the circular linked list is shown below:



Circular linked list

- A circular linked list is a sequence of elements in which each node has a link to the next node, and the last node is having a link to the first node.
- The representation of the circular linked list will be similar to the singly linked list, as shown below:

```
struct node
{
    int data;
    struct node *next;
}
```


Circular linked list

Advantages of Circular Linked List

1. In circular linked list the **next pointer** of last node points to the head node. Hence we can move from last node to the head node of the list **very efficiently**.
2. **Accessing of any node** is much **faster** than singly linked list.
3. Any node can be a **starting point**. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.
4. Useful for **implementation of queue**. We don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.

Circular linked list

5. Circular lists are useful in **applications to repeatedly** go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application.

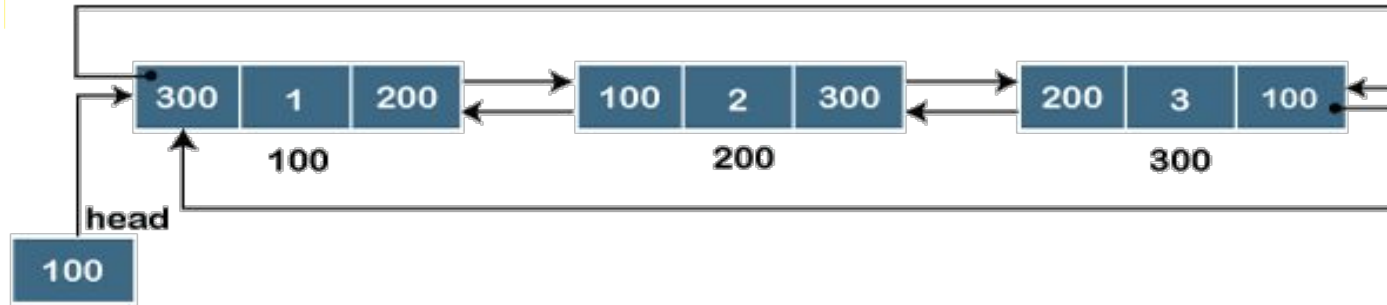
Circular linked list

Applications of Circular Linked List

1. The real life application where the circular linked list is used is our **Personal Computers**, where **multiple applications** are running. **All the running applications are kept in a circular linked list and the OS gives a fixed time slot to all for running.** The Operating System keeps on iterating over the linked list until all the applications are completed.
2. Another example can be **Multiplayer games**. All the Players are kept in a Circular Linked List and the pointer keeps on moving forward as a player's chance ends.
3. Circular Linked List can also be used **to create Circular Queue**. In a Queue we have to keep two pointers, FRONT and REAR in memory all the time, where as in Circular Linked List, only one pointer is required.

Doubly Circular linked list

- The doubly circular linked list has the features of both the circular linked list and doubly linked list.



- The main difference between the doubly linked list and doubly circular linked list is that the doubly circular linked list **does not contain the NULL value** in the previous field of the node.
- As the doubly circular linked contains three parts, i.e., two address parts and one data part so its representation is similar to the doubly linked list.

Doubly Circular linked list

```
struct node
{
    int data;
    struct node *next;
    struct node *prev;
}
```

Operations on linked list

The basic operations to be performed on the linked lists are as follows:

- 1.Creation** - This operation is used to create a linked list. Node is created when it is required and linked to the list to preserve the integrity of the list.
- 2.Searching** - This operation is used to search the item in the linked list from the starting node to end node. If the desired element is found, we signal operation **"SUCCESSFULL"**. Otherwise, we signal it as **"UNSUCCESSFULL"**.
- 3.Insertion** - This operation is used to insert a new node in the linked list at the specified position. A new node may be inserted
 - i. At the beginning of a linked list.
 - ii. At the end of a linked list.
 - iii. At the specified position in a linked list.
 - iv. If the list itself is empty, then the new node is inserted as a first node.

Operations on linked list

4. **Concatenation** - It is the process of appending (joining) the second list to the end of the first list consisting of m nodes. When we concatenate two lists, the second list has n nodes, and then the concatenated list will be having $(m + n)$ nodes. The last node of the list is modified so that it is now pointing to the first node in the second list.
5. **Deletion** - This operation is used to delete an item (a node) from the linked list. A node may be deleted from the
 - i. Beginning of a linked list
 - ii. End of a linked list
 - iii. Specified position in the list.

Operations on linked list

6. **Display** - This operation is used to print each and every node's information. We access each node from the beginning (or the specified position) of the list and output the data housed there.
7. **Traversing** - It is a process of going through all the nodes of a linked list from one end to the other end. If we start traversing from the very first node towards the last node, it is called forward traversing.

Stack

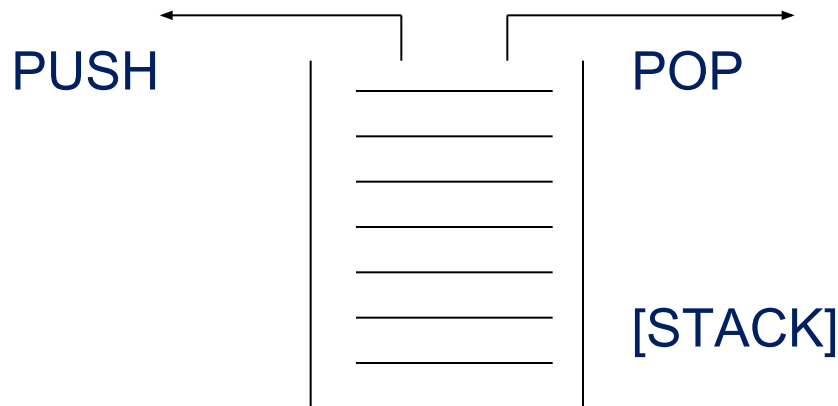
- A stack is also an ordered collection of elements like arrays, but it has a special feature that **deletion and insertion** of elements can be done only from **one end** called the top of the stack (**TOP**)
- Due to this property it is also called as **last in first out** type of data structure (**LIFO**).

Stack

- It could be thought of just like a stack of plates placed on **table in a party**, a guest always takes off a fresh plate from the top and the new plates are placed on to the stack at the top.
- It is a **non-primitive data structure**.
- When an element is inserted into a stack or removed from the stack, its base remains **fixed** where the top of stack changes.

Stack

- Insertion of element into stack is called **PUSH** and deletion of element from stack is called **POP**.
- The bellow show figure how the operations take place on a stack:

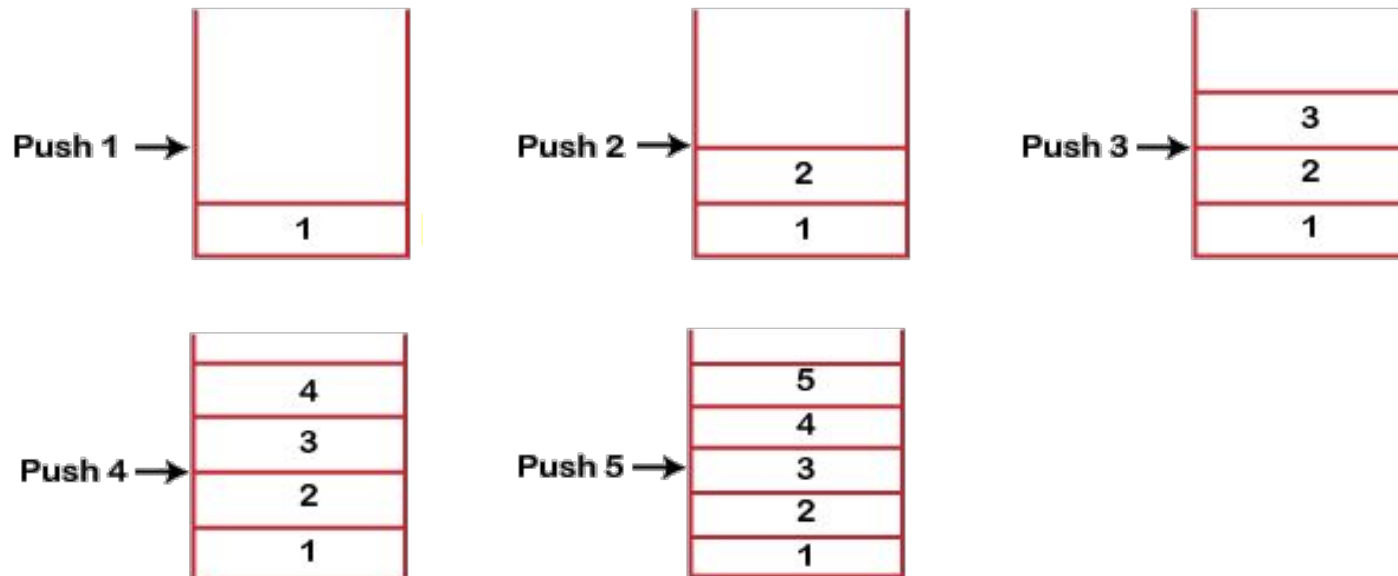


Stack

- The stack can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using pointer (Dynamic implementation)

Working of Stack

- Stack works on the LIFO pattern. As we can observe in the below figure there are five memory blocks in the stack; therefore, the size of the stack is 5.
- Suppose we want to store the elements in a stack and let's assume that stack is empty. We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.



Working of Stack

- Since our stack is full as the size of the stack is 5. In the above cases, we can observe that it goes from the top to the bottom when we were entering the new element in the stack. The stack gets filled up from the bottom to the top.
- When we perform the delete operation on the stack, there is only one way for entry and exit as the other end is closed. It follows the LIFO pattern, which means that the value entered first will be removed last. In the above case, the value 5 is entered first, so it will be removed only after the deletion of all the other elements.

Standard Stack Operations

- The following are some common operations implemented on the stack:
- `push()`: When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- `pop()`: When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- `isEmpty()`: It determines whether the stack is empty or not.
- `isFull()`: It determines whether the stack is full or not.'
- `peek()`: It returns the element at the given position.
- `count()`: It returns the total number of elements available in a stack.
- `change()`: It changes the element at the given position.
- `display()`: It prints all the elements available in the stack.

Stack

Applications:

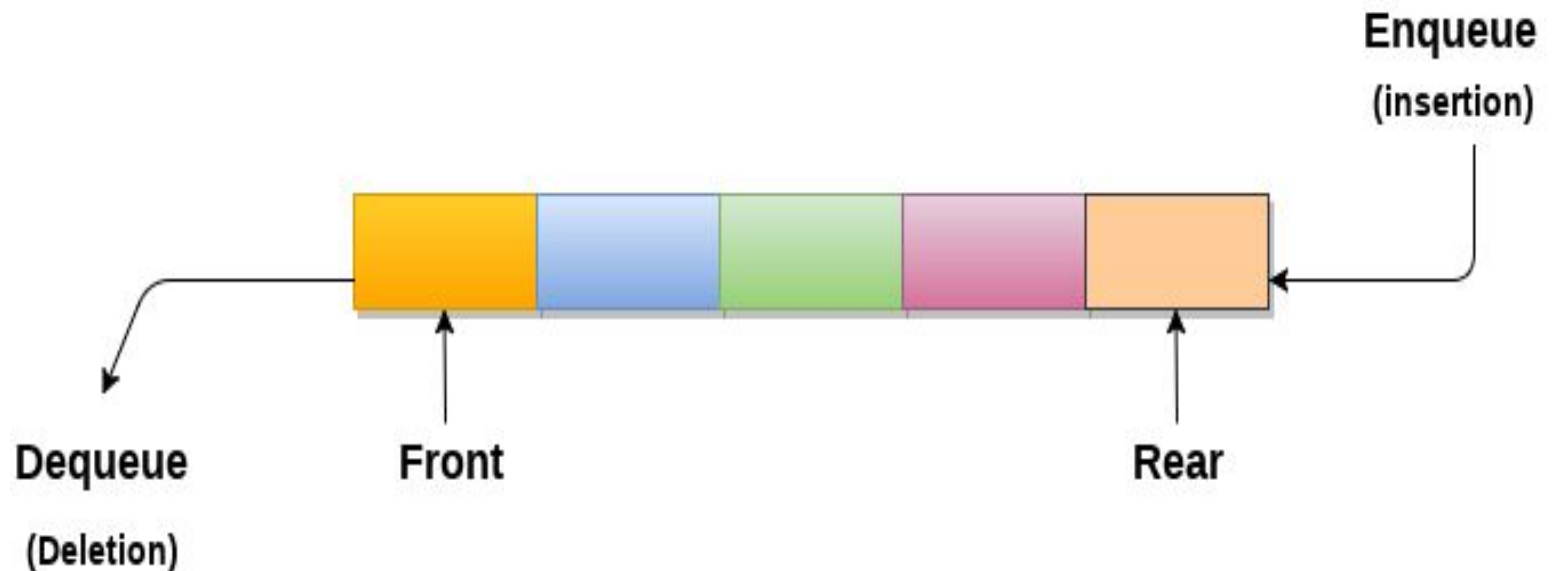
- Temporary storage structure for recursive operations
- Auxiliary storage structure for nested operations, function calls, deferred/postponed functions
- Manage function calls
- Evaluation of arithmetic expressions in various programming languages
- Conversion of infix expressions into postfix expressions
- Checking syntax of expressions in a programming environment
- Matching of parenthesis
- String reversal
- In all the problems solutions based on backtracking.
- Used in depth first search in graph and tree traversal.
- Operating System functions
- UNDO and REDO functions in an editor.

Queue

- Queue are **first in first out type** of data structure (i.e. **FIFO**)
- In a queue new elements are added to the queue from one end called **REAR** end and the element are always removed from other end called the **FRONT** end.
- The people standing in a **railway reservation** row are an example of queue.

Queue

- Each new person comes and stands at the end of the row and person getting their reservation confirmed get out of the row from the front end.
- The bellow show figure how the operations take place on a stack:



Queue

- The queue can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using pointer (Dynamic implementation)

Queue

Applications:

- It is used in **breadth search** operation in graphs.
- **Job scheduler** operations of OS like a print buffer queue, keyboard buffer queue to store the keys pressed by users
- Job scheduling, CPU scheduling, Disk Scheduling
- Priority queues are used in file downloading operations in a browser
- Data transfer between peripheral devices and CPU.
- Interrupts generated by the user applications for CPU
- Calls handled by the customers in BPO

Trees

- A tree can be defined as **finite set of data items (nodes)**.
- Tree is **non-linear** type of data structure in which data items are arranged or stored in a sorted sequence.
- Tree represent the **hierarchical relationship** between various elements.

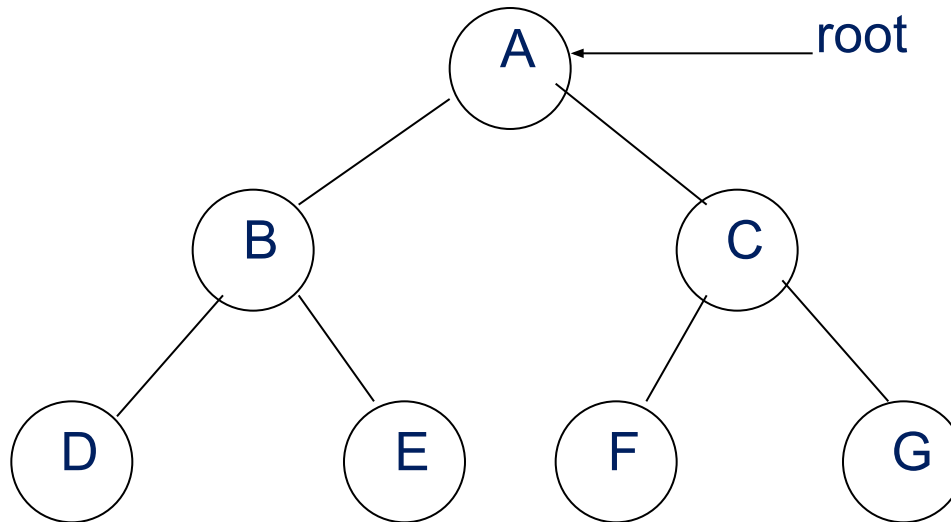
Trees

In trees:

- There is a special data item at the top of hierarchy called the **Root** of the tree.
- The remaining data items are partitioned into number of **mutually exclusive subset**, each of which is itself, a tree which is called the **sub tree**.
- The tree always grows in length towards bottom in data structures, unlike natural trees which grows upwards.

Trees

- The tree structure organizes the data into branches, which related the information.



Trees

Advantage: Provides quick search, insert, and delete operations

Disadvantage: Complicated deletion algorithm

Applications:

- Implementing the hierarchical structures in computer systems like **directory and file system**.
- Implementing the **navigation structure** of a website.
- Code generation like Huffman's code.
- **Decision making** in gaming applications.
- Implementation of priority queues for priority-based **OS scheduling functions**
- Parsing of expressions and statements in programming language compilers
- For storing data keys for DBMS for **indexing**
- Spanning trees for routing decisions in computer and communications networks
- Hash trees
- path-finding algorithm to implement in AI, robotics and video games applications

Graph

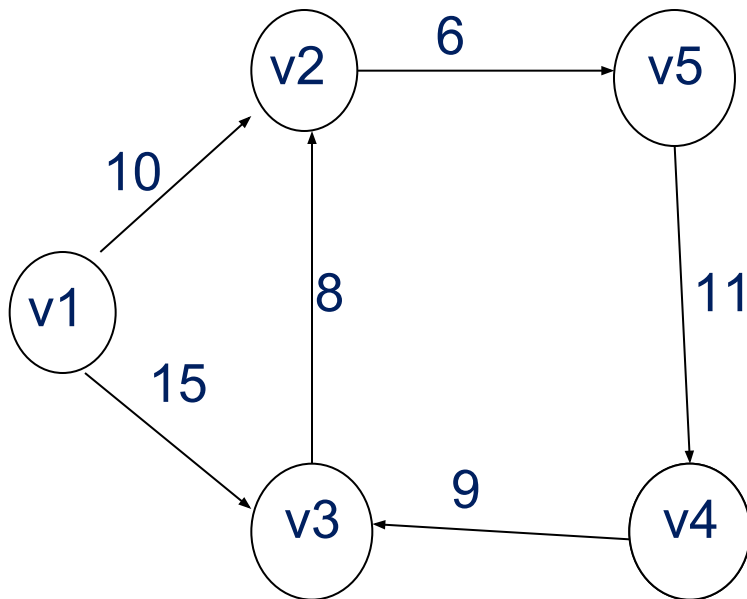
- Graph is a mathematical **non-linear data structure** capable of representing many kind of physical structures.
- It has found application in Geography, Chemistry and Engineering sciences.
- Definition: A **graph $G(V,E)$** is a set of vertices V and a set of edges E .

Graph

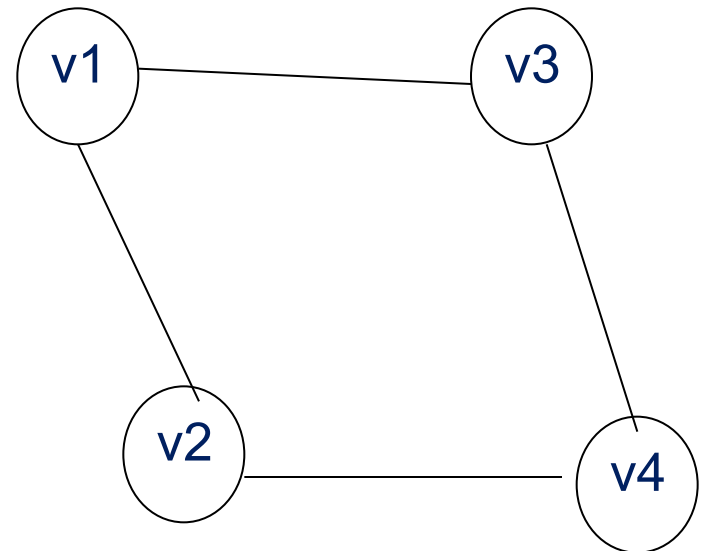
- An **edge connects a pair of vertices** and many have weight such as length, cost and another measuring instrument for according the graph.
- Vertices on the graph are shown as point or circles and edges are drawn as **arcs or line segment**.

Graph

- Example of graph:



• [a] Directed & Weighted Graph



[b] Undirected Graph

Graph

- Types of Graphs:
 - Directed graph
 - Undirected graph
 - Simple graph
 - Weighted graph
 - Connected graph
 - Non-connected graph

Graph

Advantage: Best models real-world situations

Disadvantage: Some algorithms are slow and very complex

Applications:

- Representing **networks and routes** in communication, transportation and travel applications
- **Routes in GPS**
- Interconnections in **social networks** and other network-based applications
- Mapping applications
- **Ecommerce applications** to present user preferences
- Utility networks to identify the problems posed to municipal or local corporations
- Resource utilization and availability in an organization
- Document link map of a website to display connectivity between pages through hyperlinks
- **Robotic motion** and neural networks

END